JAVA

COMPLETE NOTES

BEGINNER TO ADVANCED

Core Java

OOPs

Collections

Exception Handling

Multithreading

File Handling

Java

Learn

Practice

Improve

Grow

Introduction to Java

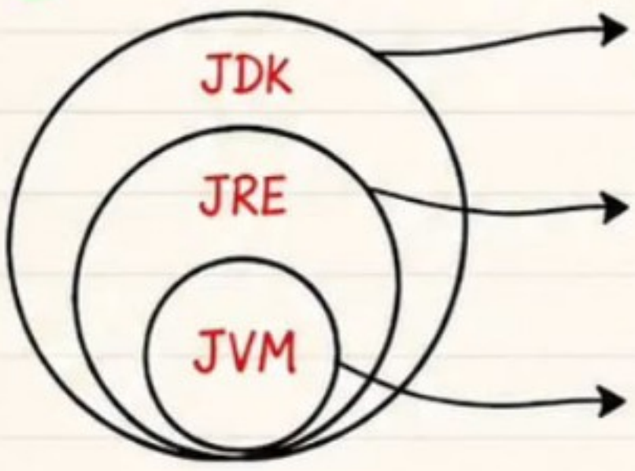
① What is Java?

Java is a high-level, class-based, object-oriented programming language designed by Sun Microsystems (now owned by Oracle). It is platform independent, secure, robust and widely used for building enterprise applications, Android apps, web apps and more.

② Features of Java

- ✓ Simple
- ✓ Object-Oriented (OOP)
- ✓ Platform Independent
- ✓ Secure
- ✓ Robust
- ✓ Multithreaded
- ✓ Architecture Neutral
- ✓ Portable
- ✓ High Performance
- ✓ Distributed
- ✓ Dynamic
- ✓ Automatic Memory Management (GC)

③ JDK vs JRE vs JVM



JDK (Java Development Kit)
Provides tools for development like javac, java, debugger, etc.

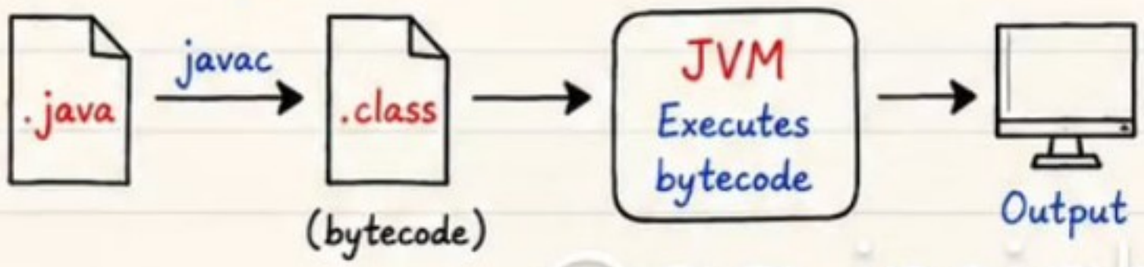
JRE (Java Runtime Environment)
Provides libraries, JVM and other components to run Java programs.

JVM (Java Virtual Machine)
Executes bytecode and provides runtime environment.

Component	Contains
JDK	JRE + JVM + Development Tools (javac, jar, javadoc, etc.)
JRE	JVM + Core Libraries + Other Runtime Components
JVM	Class Loader + Bytecode Verifier + Execution Engine + Runtime Data Areas

④ How Java Works?

Java code (.java) is compiled into bytecode (.class) using the Java Compiler (javac). This bytecode is executed by JVM on any platform.



⑤ Compile & Execution Flow

1. Write Java code (.java)
2. Compile using javac (Compiler)
3. Generates bytecode (.class file)
4. JVM loads and verifies bytecode
5. JVM executes bytecode
6. Output is displayed

@qa_insights

⑥ First Java Program ★

```
// Hello World Program
public class HelloWorld {
    public static void main(String[] args) {
        System.out.println("Hello, Java!");
    }
}
```

- class keyword → defines a class
- main() → entry point of program
- System.out.println() → prints output
- Every statement ends with ;
- File name must match class name (HelloWorld.java)

✓ Remember

- Java is "Write Once, Run Anywhere" (WORA).
- JVM makes Java platform independent.
- Bytecode is the heart of Java.

★ Interview Fact

Java was designed by James Gosling in 1995 and first released in 1996. The file extension for Java source code is .java and for bytecode is .class

⚠ Common Mistake

- Filename and class name mismatch.
- Missing main() method.
- Not understanding compilation vs execution.

JDK, JRE & JVM

① JDK (Java Development Kit)

- Full package for Java development.
- Includes JRE + development tools.
- Used to write, compile and run Java programs.

Includes:
- JRE
- Compiler (javac)
- Tools (javadoc, jar, jdb, etc.)

② JRE (Java Runtime Environment)

- Provides libraries, JVM and other components to run Java applications.
- Does not include development tools.

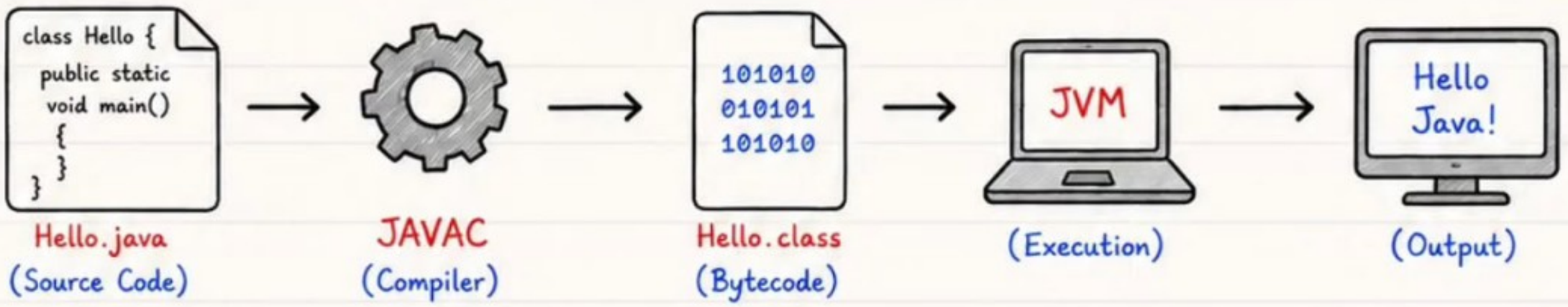
Includes:
- JVM
- Core Libraries (rt.jar, etc.)

③ JVM (Java Virtual Machine)

- Heart of Java.
- Executes Java bytecode.
- Platform independent - Write Once, Run Anywhere.
- Manages memory, threads, security and more.

@qa_insights

④ Compilation Process



⑤ Bytecode

- Output of Java compiler (javac).
- Platform independent.
- Executed by JVM, not by OS directly.
- File extension: .class

Important Note

JVM converts bytecode into machine code at runtime using JIT (Just-In-Time) compiler for better performance.

⑥ Execution Flow Diagram

Write Java Code (.java) → Compile Using JAVAC → Generate Bytecode (.class) → JVM Loads Bytecode & Verifies → JVM Executes Bytecode (Interpretation + JIT) → Output on Console

☆ Java is Platform Independent because JVM is available on all major OS.

Key Takeaway

- ☑ Develop in JDK
- ☑ Run in JRE
- ☑ Execution happens inside JVM
- ☑ Write Once, Run Anywhere

Variables & Data Types

① What is a Variable?

A variable is a named location in memory used to store data that can be changed during program execution.

```
int   age  =  25;
```

Data Type — Variable Name — Value

② Data Types

a) Primitive Types

Store single values.

Type	Size	Default Value
byte	1 byte	0
short	2 bytes	0
int	4 bytes	0
long	8 bytes	0L
float	4 bytes	0.0f
double	8 bytes	0.0d
char	2 bytes	'\u0000'
boolean	1 bit	false

b) Non-Primitive Types

Store reference to objects.

- String
- Array
- Class
- Interface
- Object
- Collection (List, Set, Map)
- etc.

③ Literals

- Integer Literal : 25, -10, 0
- Floating Literal : 3.14, -0.5f
- Character Literal : 'A', 'z', '\n'
- String Literal : "Hello Java"
- Boolean Literal : true, false

④ Naming Rules for Variables

- ☑ Must start with a letter, '$' or '_'
- ☑ Can contain letters, digits, '_' and '$'
- ☑ Case sensitive: age and Age are different
- ☑ Cannot be a Java keyword (int, class, etc.)
- ☒ Examples of invalid names: 1age, int, @age

⑤ Memory Basics

	Memory	
1001 →	25	→ int age = 25;
1002 →	3.14	→ double pi = 3.14;
1003 →	'A'	→ char grade = 'A';
1004 →	true	→ boolean isActive = true;
	...	

Note:
Each variable occupies memory based on its data type.
Primitive types store actual values.
Non-primitive types store reference (address) to objects in memory.

Interview Tip

```
int a = 10;
int b = a;  // b gets a copy of value
a = 20;     // b remains unchanged
System.out.println(b); // Output: 10
```

Best Practices

- ☑ Use meaningful variable names.
- ☑ Declare variables close to where they are used.
- ☑ Prefer final for constants.
- ☑ Follow camelCase naming convention.

Operators & Type Casting

① Arithmetic Operators

Used to perform basic mathematical operations.

Operator	Description	Example
+	Addition	a + b
-	Subtraction	a - b
*	Multiplication	a * b
/	Division	a / b
%	Modulus (Remainder)	a % b
++	Increment	a++
--	Decrement	a--

② Relational Operators

Used to compare two values. Result is boolean (true/false).

Operator	Description	Example
==	Equal to	a == b
!=	Not equal to	a != b
>	Greater than	a > b
<	Less than	a < b
>=	Greater than or equal	a >= b
<=	Less than or equal	a <= b

③ Logical Operators

Used to combine multiple conditions.

Operator	Description	Example
&&	Logical AND	(a > 0) && (b > 0)
||	Logical OR	(a > 0) || (b > 0)
!	Logical NOT	!(a > 0)

④ Assignment Operators

Used to assign values to variables.

Operator	Example	Same As
=	a = b	a = b
+=	a += b	a = a + b
-=	a -= b	a = a - b
*=	a *= b	a = a * b
/=	a /= b	a = a / b
%=	a %= b	a = a % b

⑤ Unary Operators

Operate on a single operand.

Operator	Description	Example
+	Unary Plus	+a
-	Unary Minus	-a
++	Pre-Increment	++a
--	Pre-Decrement	--a
!	Logical Complement	!true → false
~	Bitwise Complement	~a

⑥ Type Casting

Converting one data type into another.

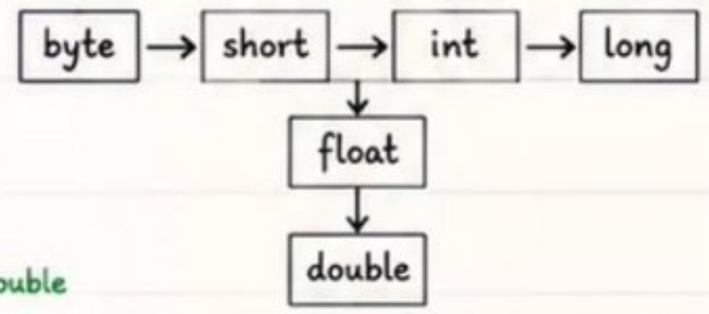
- Implicit (Widening) - done automatically.
- Explicit (Narrowing) - done manually.

1) Implicit (Widening)

Smaller → Larger type

Example:

```
int a = 10;
double b = a;  // int to double
```

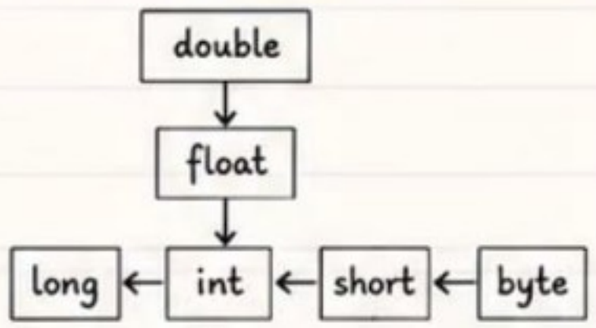


2) Explicit (Narrowing)

Larger → Smaller type (Manual)

Example:

```
double x = 25.75;
int y = (int) x;  // double to int
```



Example Code

```
int a = 10, b = 3;
System.out.println("a + b = " + (a + b));   // 13
System.out.println("a - b = " + (a - b));   // 7
System.out.println("a * b = " + (a * b));   // 30
System.out.println("a / b = " + (a / b));   // 3
System.out.println("a % b = " + (a % b));   // 1
System.out.println("a > b : " + (a > b));   // true
```

Interview Tip

Operators are the building blocks of expressions and conditions in Java. Understand precedence and type casting to avoid bugs.

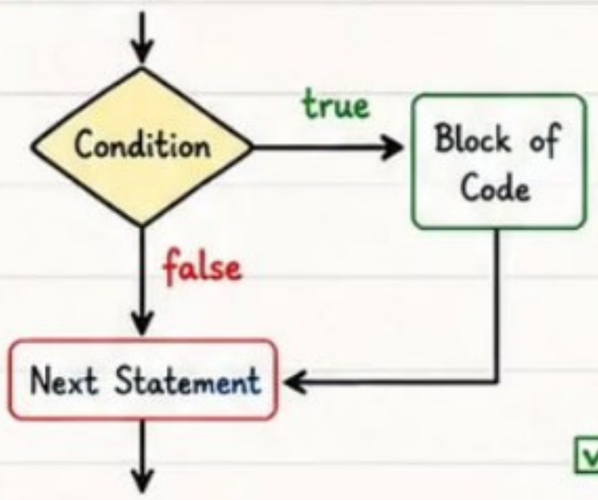
Common Mistake

Always be careful while type casting.
Narrowing conversions may lead to data loss.

Control Statements

Control statements change the flow of execution based on conditions or perform repeated execution of a block of code.

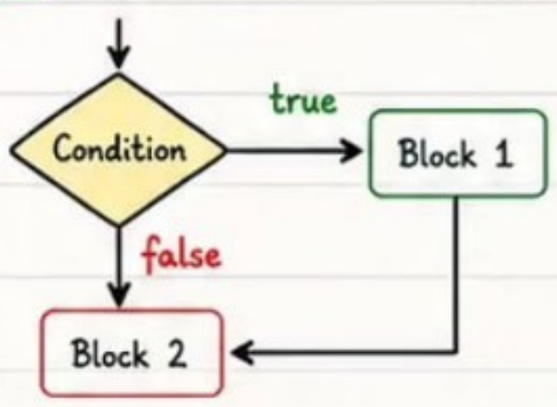
① if




```
int age = 18;
if (age >= 18) {
    System.out.println(
    "Eligible to Vote");
}
```

☑ Executes block if condition is true.

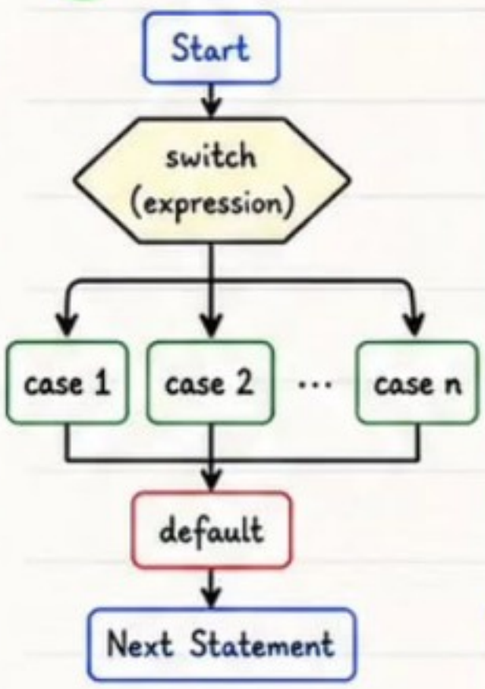
② if-else




```
int n = -5;
if (n >= 0) {
  System.out.println("Positive");
} else {
  System.out.println("Negative");
}
```

☑ Executes one block from two based on condition.

③ switch

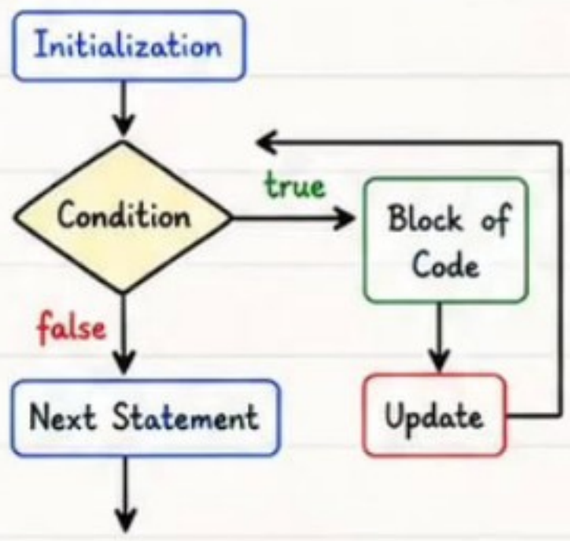



```
int day = 2;
switch (day) {
 case 1 -> System.out.println("Mon");
 case 2 -> System.out.println("Tue");
 case 3 -> System.out.println("Wed");
 default -> System.out.println("Invalid");
}
```

☑ Used when we have multiple possible values to match.

Note:
In Java 17+, switch supports arrow (->) syntax and works with String, enum also.

④ for

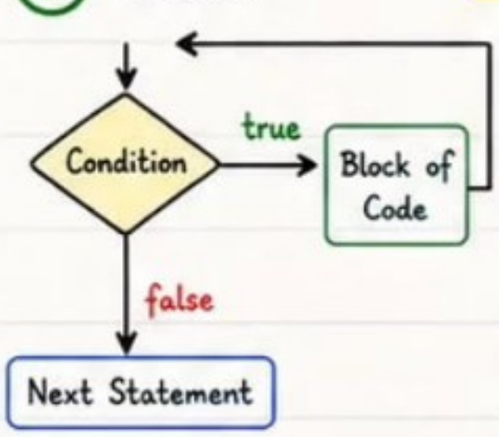



```
for (int i = 1; i <= 5; i++) {
  System.out.println(i);
}
```

☑ Best when number of iterations is known.

@qa_insights

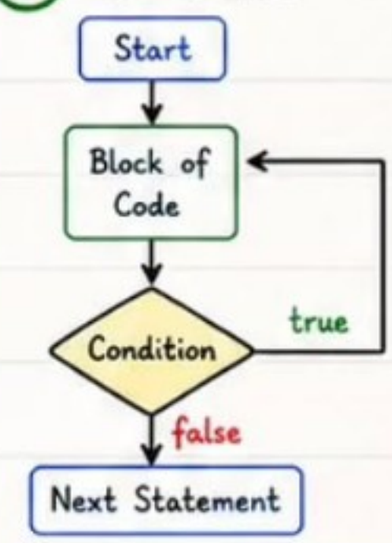
⑤ while




```
int i = 1;
while (i <= 5) {
  System.out.println(i);
  i++;
}
```

☑ Executes block while condition is true.

⑥ do-while




```
int i = 1;
do {
  System.out.println(i);
  i++;
} while (i <= 5);
```

☑ Executes block at least once, then checks condition.

⑦ break & continue

- **break**
  Terminates the loop or switch immediately.

```
for (int i = 1; i <= 5; i++) {
  if (i == 3) break;
  System.out.print(i + " ");
}
```

☆ Output: 1 2

- **continue**
  Skips current iteration and moves to next iteration.

```
for (int i = 1; i <= 5; i++) {
  if (i == 3) continue;
  System.out.print(i + " ");
}
```

☆ Output: 1 2 4 5

🎯 **Interview Tip:** Choose the right control statement to improve code readability and performance.

☑ **Best Practice:** Avoid deep nesting, Use early return, break, or continue to simplify logic.

Methods in Java

A method is a block of code that performs a specific task.

① Method Syntax

```
accessModifier returnType methodName(parameterList) {
    // method body
    return value;
}
```

- public, private protected, etc.
- int, void, String, etc.
- Name of the method
- Input values

② Parameters

- Parameters are used to pass data into a method.
- They act as placeholders.

```
public int add(int a, int b) {
    return a + b;
}
```

a and b are parameters

③ Return Types

- Method can return a value to the caller using return statement.

Return Type	Meaning
void	No value is returned
int, double, etc.	Returns numeric value
String	Returns text value
boolean	Returns true or false

Example

```
public int square(int n){
    return n * n;
}
```

④ Method Overloading

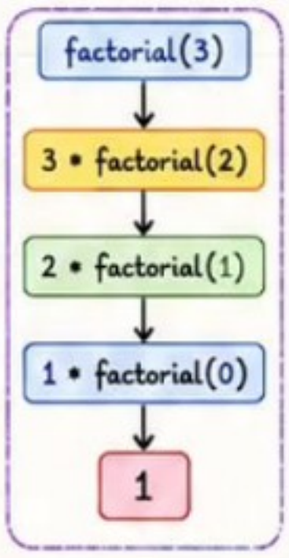
- Same method name with different parameters.
- Improves code readability.

```
int add(int a, int b) {
    return a + b;
}
int add(int a, int b, int c) {
    return a + b + c;
}
```

⑤ Recursion

- A method calling itself.
- Used to solve complex problems in a simple way.

```
int factorial(int n) {
  if (n == 0) return 1;    // Base Case
  return n * factorial(n - 1); // Recursive Call
}
```



⑥ Variable Scope

Type	Scope	Lifetime	Example
Local Variable	Inside method or block	Created when method/block is called	int x = 10;
Instance Variable (Non-static)	Inside class, outside methods	Exists as long as object exists	int age;
Static Variable (Class Variable)	Whole class	Exists till program ends	static int count;

⑦ Method Example (Complete Flow)

```
public class Demo {
    public static int multiply(int a, int b) {
        return a * b;
    }
    public static void main(String[] args) {
      int result = multiply(5, 4); // method call
      System.out.println("Result: " + result);
    }
}
```

How it works?

main() Method Called → multiply() Method Executed → Returns Result → Result Used in main()

Interview Tip

Methods help in code reusability, maintainability and modular programming.

Best Practices

- ☑ Use meaningful method names.
- ☑ Keep methods small and focused.
- ☑ Avoid duplicate code.
- ☑ Follow single responsibility principle.

Remember:

A good method makes your code clean and easy to understand.

Arrays in Java

An array is a collection of similar data types stored in contiguous memory locations.

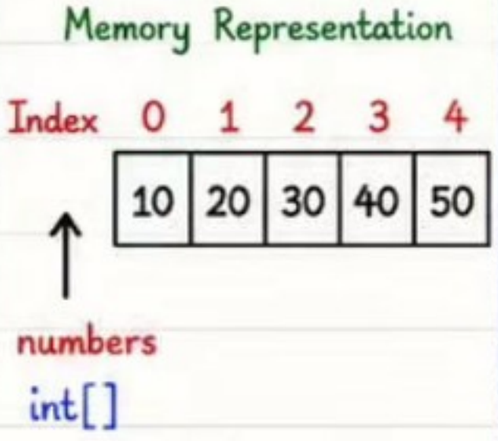
① 1D Arrays

- Stores elements of the same type in a single row.
- Index starts from 0.

Example:

```
int[] numbers = new int[5];
numbers[0] = 10;
numbers[1] = 20;
numbers[2] = 30;
numbers[3] = 40;
numbers[4] = 50;
```

Memory Representation



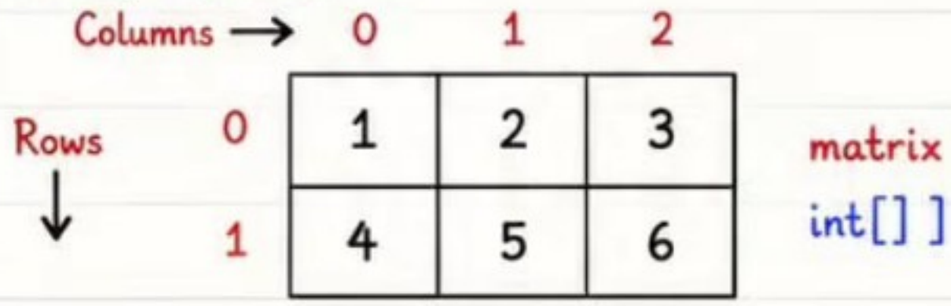
② 2D Arrays

- Array of arrays (rows and columns).
- Useful for tables, matrices, grids.

Example:

```
int[][] matrix = new int[2][3];
matrix[0][0] = 1;  matrix[0][1] = 2;  matrix[0][2] = 3;
matrix[1][0] = 4;  matrix[1][1] = 5;  matrix[1][2] = 6;
```

Memory Representation



③ Array Operations

Operation	Description	Example
Declare	Create an array	`int[] arr = new int[5];`
Access	Access element using index	`int val = arr[2];`
Update	Update element at index	`arr[2] = 100;`
Length	Get total number of elements	`int len = arr.length;`
Traversal	Loop through all elements	`for (int i = 0; i < arr.length; i++) { ... }`

④ for-each Loop

- Simplified way to traverse arrays.
- Read-only access (cannot modify index).

Example:

```
int[] arr = {10, 20, 30, 40, 50};
for (int num : arr) {
    System.out.println(num);
}
```

☆ Output

```
10
20
30
40
50
```

⑤ Arrays Utility Class

- java.util.Arrays provides useful methods to work with arrays.

Method	Description
Arrays.toString(arr)	Returns string representation
Arrays.sort(arr)	Sorts array in ascending order
Arrays.fill(arr, val)	Fills array with given value
Arrays.copyOf(arr, newLen)	Copies array to new length
Arrays.equals(arr1, arr2)	Compares two arrays

Interview Tip

- Arrays are fixed in size.
- IndexOutOfBoundsException occurs for invalid index.
- Use for-each loop for simple traversal.

Best Practices

- Use meaningful names.
- Always check index before accessing.
- Prefer Arrays utility methods for operations.
- Avoid storing large data in stack, use arrays

Strings in Java

① String

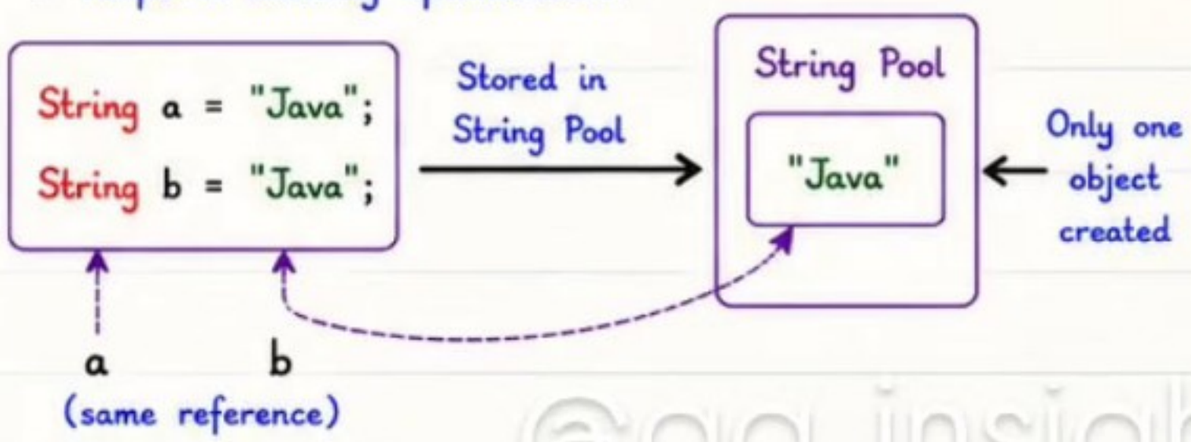
- A String is a sequence of characters.
- In Java, String is an object of class java.lang.String.
- Strings are widely used to store and manipulate text.

Example:

```
String name = "Java";
String msg = new String("Hello");
System.out.println(name);     // Java
System.out.println(msg);      // Hello
```

② String Pool

- String literals are stored in the String Pool.
- If a literal already exists, the same reference is returned.
- Helps in memory optimization.



③ Immutable Strings

- String objects are immutable (cannot be changed).
- Any operation that seems to modify the string actually creates a new object.

Example:

```
String s1 = "Hello";
String s2 = s1.concat(" World");
System.out.println(s1);   // Hello
System.out.println(s2);   // Hello World
```

s1 is unchanged.
A new object is created for s2.

④ StringBuilder

- Mutable class, used to modify strings.
- Not synchronized → not thread-safe.
- More efficient than String for single-threaded environments.

Example:

```
StringBuilder sb = new StringBuilder("Java");
sb.append(" Programming");
sb.insert(5, " is");
sb.delete(0, 1);                // removes 'J'
System.out.println(sb);         // ava is Programming
```

⑤ StringBuffer

- Mutable class like StringBuilder.
- Synchronized → thread-safe.
- Slower than StringBuilder but safe for multi-threading.

Example:

```
StringBuffer sb = new StringBuffer("Java");
sb.append(" Programming");
sb.reverse();
System.out.println(sb);     // gnimmargorP avaJ
```

⑥ String vs StringBuilder vs StringBuffer

Feature	String	StringBuilder	StringBuffer
Mutable?	No	Yes	Yes
Thread Safe?	Yes	No	Yes
Performance	Slow	Fast	Slow
Use Case	Less modification	Single-threaded performance critical	Multi-threaded environments

⑦ Common String Methods

Method	Description	Example	Output
length()	Returns length of string	"Java".length()	4
charAt(index)	Returns character at index	"Java".charAt(1)	a
substring(begin, end)	Returns part of string [begin, end)	"Java".substring(1, 3)	av
equals(str)	Compares content	"Java".equals("java")	false
equalsIgnoreCase(str)	Compares ignoring case	"Java".equalsIgnoreCase("java")	true
contains(str)	Checks if sequence exists	"Java Programming".contains("Prog")	true
indexOf(str)	First index of sequence	"Java".indexOf('v')	2
replace(old, new)	Replaces all occurrences	"Java".replace('a', 'o')	Jovo
toUpperCase()	Converts to upper case	"Java".toUpperCase()	JAVA
trim()	Removes leading & trailing spaces	" Java ".trim()	"Java"

Interview Tip

- Strings are immutable and stored in String Pool.
- Use StringBuilder for better performance.
- Use StringBuffer when thread safety is required.

Best Practices

- Prefer StringBuilder over String for concatenation in loops.
- Avoid creating unnecessary String objects.
- Use meaningful variable names for better readability.

Classes & Objects in Java

Java is an object-oriented programming language.
Everything in Java is an Object.

① Class

- A Class is a blueprint or template.
- It defines the properties (fields) and behaviors (methods) of objects.
- It does not occupy memory.

Example: Class Definition

```
class Student {
    String name;          // field
    int age;              // field

    void display() {      // method
        System.out.println(name + " is " + age);
    }
}
```

② Object

- An Object is a real-world entity.
- It is an instance of a Class.
- It occupies memory in Heap.

Example: Objects of Student Class

s1 (Object)	
name	"Alice"
age	20

s2 (Object)	
name	"Bob"
age	22

Both objects are instances of Student class

③ Fields (Variables)

- Fields represent the state or data of an object.
- Each object has its own copy of fields.

④ Methods

- Methods represent the behavior of an object.
- Methods operate on fields and perform actions.
- Methods are defined inside the class.

⑤ Object Creation

- Objects are created using the new keyword.

Syntax:

```
ClassName objectName = new ClassName();
```

Example:

```
Student s1 = new Student();    // s1 is an object
Student s2 = new Student();    // s2 is another object
```

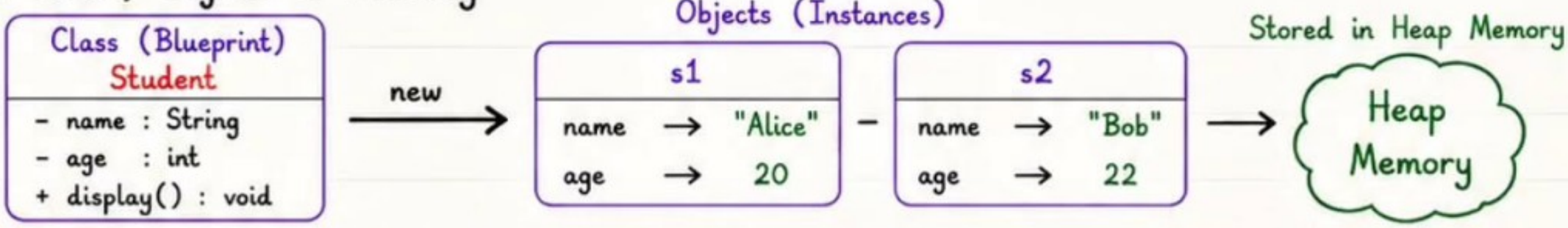
⑥ Accessing Members

- Use dot (.) operator to access fields and methods of an object.

Example:

```
Student s1 = new Student();
s1.name = "Alice";             // access field
s1.age  = 20;                  // access field
s1.display();                  // call method
```

⑦ Class, Object & Memory



Interview Tip

- Class is a template, Object is an instance.
- Multiple objects can be created from one class.
- Memory for objects is allocated in Heap.

Best Practices

- Design classes around real-world entities.
- Keep fields private and provide public methods (encapsulation).
- Use meaningful names for classes, fields and methods.

Constructors & Keywords in Java

① Constructor

- A Constructor is a special method used to initialize objects.
- It has the same name as the class.
- It has no return type (not even void).
- It is called automatically when an object is created.

Syntax:

```
class ClassName {
    ClassName() {
        // initialization code
    }
}
```

Note

Constructors are used to set initial values for object properties.

② Default Constructor

- A constructor with no parameters.
- If we don't create any constructor, Java provides a default constructor automatically.
- It initializes object with default values.

Example:

```
class Student {
    String name;
    int age;

    // Default constructor (provided by Java)
    Student() { }
}
```

③ Parameterized Constructor

- A constructor that takes parameters.
- Used to initialize object with specific values.

Example:

```
class Student {
    String name;
    int age;

    Student(String name, int age) {
        this.name = name;
        this.age  = age;
    }
}
```

④ Keyword: this

- this is a reference variable that refers to the current object.
- Used to differentiate instance variables from local variables.

Example:

```
class Student {
    String name;
    int age;

    Student(String name, int age) {
        this.name = name;  // refers to instance variable
        this.age  = age;   // refers to instance variable
    }
}
```

⑤ Keyword: static

- static members belong to the Class, not to any object.
- Can be accessed using class name.
- Shared among all objects.

Example:

```
class Student {
    static int count = 0;    // class variable
    String name;

    Student(String name) {
        this.name = name;
        count++;             // same for all objects
    }
}
```

⑥ Keyword: final

- final variable: value cannot be changed.
- final method: cannot be overridden.
- final class: cannot be inherited.

Example:

```
class Student {
    final String COLLEGE = "ABC Institute"; // constant

    final void show() {
    }   System.out.println("This method cannot be overridden");
}
```

Quick Comparison

Keyword	Usage	Belongs to	Can be Changed?
this	Refers to current object	Instance	Yes
static	Class level members	Class	Only once
final	Constant / Restrict changes & inheritance	Depends (variable/method/class)	No

Interview Tip

- Constructor name must match the class name.
- Constructor has no return type.
- Default constructor is created by JVM if none is defined.

Best Practices

- Use this to avoid ambiguity between variables.
- Use static for shared data and utility methods.
- Use final to make code secure and immutable.

Encapsulation & Inheritance in Java

① Encapsulation

- Encapsulation is the binding of data (fields) and methods that operate on the data into a single unit (class).
- It restricts direct access to some of an object's components which is a way of protecting the data.

Benefits

- Data Hiding
- Improved Security
- Better Control on Data
- Increased Flexibility

② Getters & Setters

- Used to access and update private fields.
- Getter returns the value.
- Setter validates and sets the value.

Example:

```
class Employee {
    private String name;

    public String getName() {
        return name;
    }
    public void setName(String name) {
        this.name = name;
    }
}
```

③ Inheritance

- Inheritance is the mechanism where one class acquires (inherits) the properties and behaviors of another class.
- It promotes code reusability and establishes IS-A relationship.

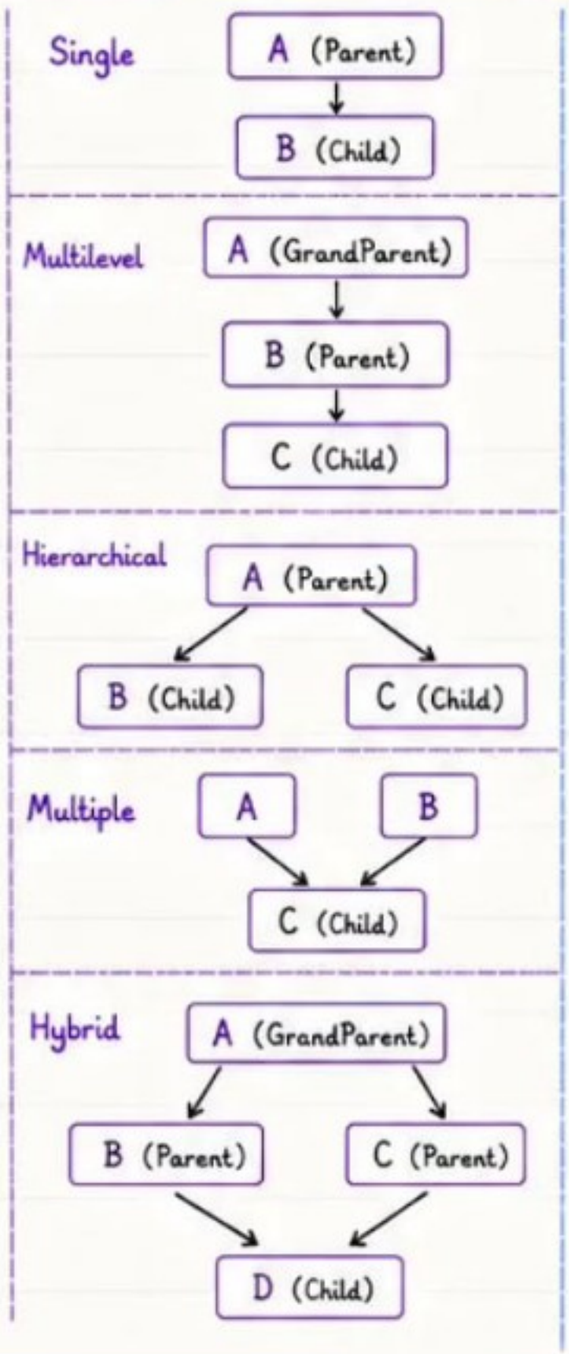
IS-A Relationship

If class B inherits from class A, then B IS-A A.

Example: Dog IS-A Animal

④ Types of Inheritance

- Single Inheritance – One child inherits from one parent.
- Multilevel Inheritance – Child inherits from parent, and another child inherits from that child.
- Hierarchical Inheritance – Multiple children inherit from one parent.
- Multiple Inheritance – One child inherits from multiple parents. (Not supported in Java with classes, supported with Interfaces)
- Hybrid Inheritance – Combination of two or more types.



⑤ super Keyword

- super is a reference variable that refers to the immediate parent class object.
- Used to access parent class members (fields, methods, constructors).

Example:

```
class Animal {
    String type = "Animal";
    Animal() { System.out.println("Animal Constructor"); }
}
class Dog extends Animal {
    String type = "Dog";
    Dog() {
        super();                    // calls parent constructor
        System.out.println("Dog Constructor");
    }
    void show() {
        System.out.println(super.type);  // accesses parent field
    }
}
```

super can be used to:

- ✓ Access parent class fields: super.fieldName
- ✓ Access parent class methods: super.methodName()
- ✓ Call parent class constructor: super();

Interview Tip

- ✓ Encapsulation protects data using private + getters/setters.
- ✓ Inheritance models real-world hierarchy using IS-A.
- ✓ Choose composition over deep inheritance when possible.

Best Practices

- ✓ Declare fields private and provide public access methods.
- ✓ Use meaningful names for classes and methods.
- ✓ Use super() in child constructor to initialize parent part.
- ✓ Prefer Interfaces over Multiple Inheritance with classes.

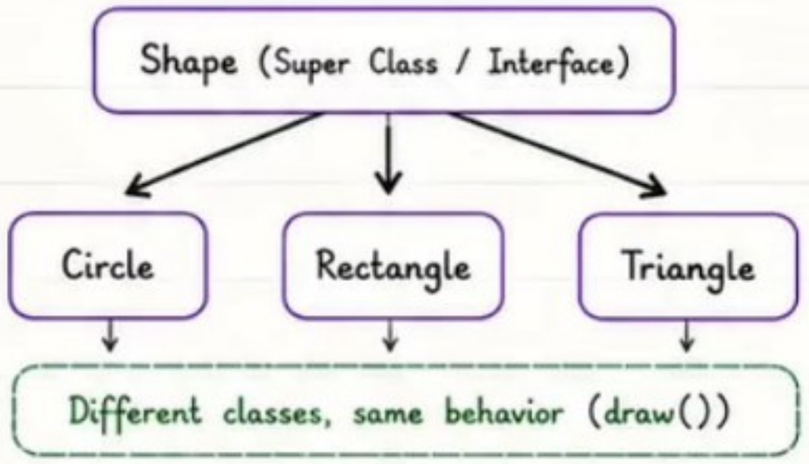
Polymorphism & Abstraction in Java

① Polymorphism

- Polymorphism means "many forms".
- It allows objects of different classes to be treated as objects of a common super class.
- Achieved using Method Overloading (compile-time) and Method Overriding (runtime).

Key Idea

Same method name, different behavior.
One interface, multiple implementations.



② Compile-time Polymorphism

- Achieved using Method Overloading.
- Same method name, different parameters.
- Resolved by the compiler at compile time.

Example:

```
class Calculator {
    int add(int a, int b) { return a + b; }
    int add(int a, int b, int c) { return a + b + c; }
    double add(double a, double b) { return a + b; }
}
```

@qa_insights

③ Runtime Polymorphism

- Achieved using Method Overriding.
- Same method name and signature in parent and child class.
- Resolved by JVM at runtime (dynamic dispatch).

Example:

```
class Animal {
    void sound() { System.out.println("Animal sound"); }
}
class Dog extends Animal {
    @Override
    void sound() { System.out.println("Bark"); }
}
Animal a = new Dog();     // Parent reference
a.sound();                // Bark (runtime polymorphism)
```

④ Abstract Class

- Can have both abstract and concrete methods.
- Cannot be instantiated.
- Used to provide a common base and partial implementation.

Example:

```
abstract class Shape {
    abstract double area();        // abstract method
    void display() {               // concrete method
        System.out.println("This is a shape");
    }
}
```

⑤ Interface

- Contains only abstract methods (before Java 8).
- From Java 8, can have default and static methods.
- Fully abstract = multiple inheritance of type.

Example:

```
interface Drawable {
    void draw();                   // abstract method
    default void info() {          // default method (Java 8+)
        System.out.println("Drawable interface");
    }
}
```

⑥ Method Overriding

- Child class provides specific implementation of parent class method.
- Must have same method name, same parameters, and same return type.
- Access level cannot be more restrictive.

Example:

```
class Parent {
    void show() { System.out.println("Parent"); }
}
class Child extends Parent {
    @Override
    void show() { System.out.println("Child"); }
}
```

Quick Comparison

Feature	Compile-time Polymorphism	Runtime Polymorphism
Also Called	Static Polymorphism	Dynamic Polymorphism
Achieved By	Method Overloading	Method Overriding
Binding Time	Compile Time	Runtime
Method Signature	Different	Same
Inheritance	Not Required	Required
Example	add(int, int)	sound() in Parent & Child

Interview Tip

- Overloading is about parameters.
- Overriding is about inheritance.
- Prefer interfaces for defining contracts.

Best Practices

- Use abstraction to hide implementation and show behavior.
- Program to interfaces, not to implementations.
- Use @Override annotation always for method overriding.

Exception Handling in Java

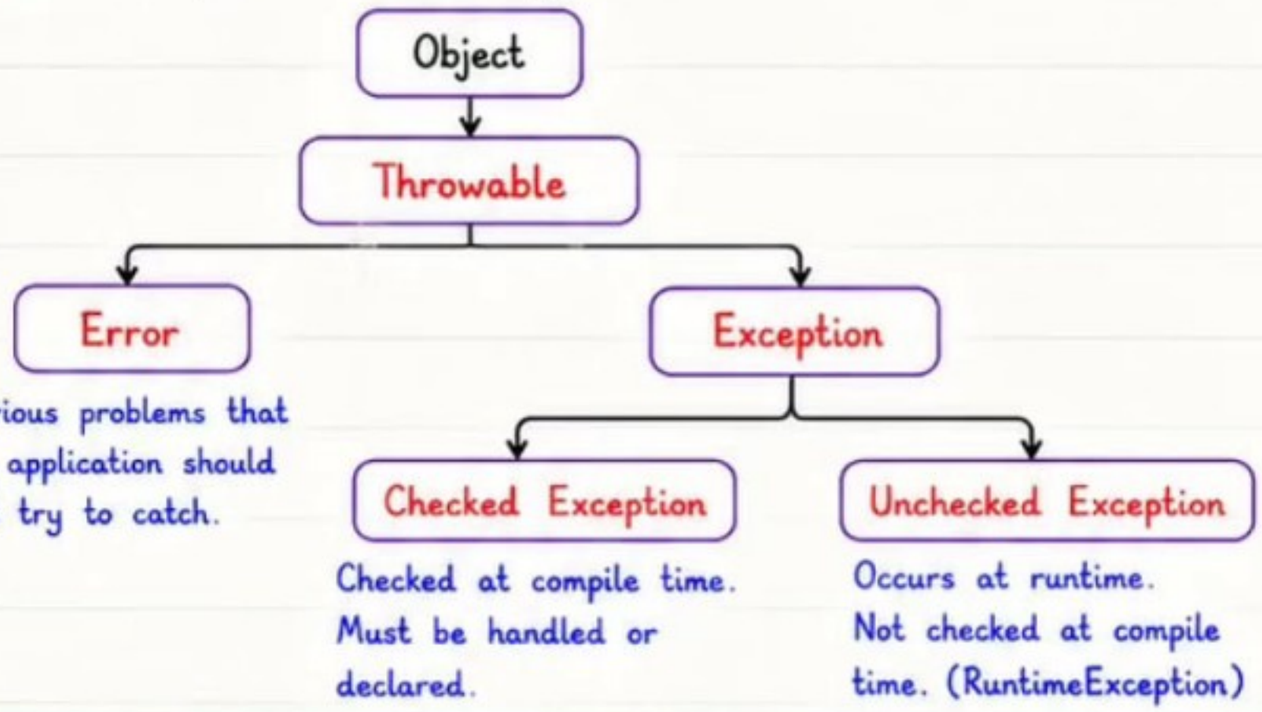
① What is Exception?

An Exception is an unwanted or unexpected event that disrupts the normal flow of a program.

Benefits

- Improves code reliability
- Avoids abnormal termination
- Separates error handling logic
- Helps in better debugging

② Exception Hierarchy



Serious problems that an application should not try to catch.

Checked at compile time. Must be handled or declared.

Occurs at runtime. Not checked at compile time. (RuntimeException)

③ try - catch - finally

- try : Contains risky code.
- catch : Handles the exception.
- finally : Always executes (optional).

```
try {
    int result = 10/0;   // risky code
} catch (ArithmeticException e) {
    System.out.println("Cannot divide by zero");
} finally {
    System.out.println("Finally block always runs");
}
```

④ throw Keyword

- Used to explicitly throw an exception.
- You can throw any Object that is a subclass of Throwable.

Example:

```
public void validate(int age) {
  if (age < 18) {
     throw new IllegalArgumentException
       ("Age must be 18 or above");
  }
}
```

⑤ throws Keyword

- Used in method signature to declare exception.
- It shifts the responsibility to the caller.

Example:

```
public void readFile() throws
  IOException {
  FileReader fr = new FileReader
   ("file.txt");
}
```

⑥ Custom Exception

- We can create our own exception by extending Exception class.

Example:

```
class InvalidAgeException extends Exception {
    public InvalidAgeException(String msg) {
        super(msg);
    }
}
// Usage
if (age < 0) {
    throw new InvalidAgeException("Age cannot
                be negative");
}
```

⑦ Common Exceptions

Exception	Description	Type
ArithmeticException	Divide by zero, invalid math operations	Unchecked
NullPointerException	Accessing methods on null object	Unchecked
ArrayIndexOutOfBoundsException	Invalid index in array	Unchecked
NumberFormatException	Invalid conversion of String to number	Unchecked
IOException	Input/Output related exceptions	Checked
ClassNotFoundException	Class not found at runtime	Checked

@qa_insights

Interview Tip

- Always prefer specific exceptions over general Exception.
- Use finally to release resources (DB, files, streams).

Best Practices

- Never ignore exceptions.
- Log meaningful error messages.
- Do not catch Exception unless necessary.
- Use custom exceptions to represent business rules.

Packages & Access Modifiers in Java

① Packages

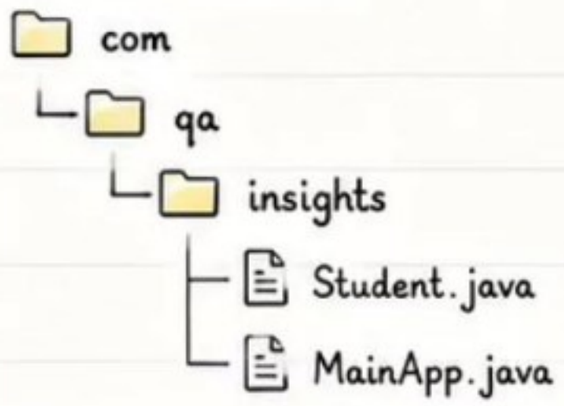
- A Package is a namespace that organizes related classes and interfaces.
- Helps in avoiding naming conflicts and improves code maintainability.
- Built-in packages: java.lang, java.util, java.io, etc.

Benefits

- ✓ Avoids name conflicts
- ✓ Better organization
- ✓ Easier code maintenance
- ✓ Access control

② Creating & Using Packages

Package Structure



Package Declaration (in Student.java)

```
package com.qa.insights;
```

Importing a Class

```
import com.qa.insights.Student;
```

③ import Statement

- Used to use classes from other packages.

Types:

- Single type import

```
import java.util.ArrayList;
```

- On-demand import

```
import java.util.*;
```

- Static import (Java 5+)

```
import static java.lang.Math.*;
```

Note

Classes from java.lang package are imported automatically.

④ Access Modifiers

Modifier	Within Class	Within Package	Subclasses (in other package)	Outside Package	Best For
public	Yes	Yes	Yes	Yes	Maximum accessibility
private	Yes	No	No	No	Data hiding
protected	Yes	Yes	Yes	No	Inheritance
default (no modifier)	Yes	Yes	No	No	Package level access

⑤ Examples

```
// Public - accessible everywhere
public class PublicClass {}

// Private - accessible only within class
class PrivateClass {
   private int secret = 10;
}
```

```
// Protected - accessible in package
// and by subclasses

class Parent {
    protected void show() {}
}

// Default - accessible only in package
class DefaultClass {}
```

⑥ Access in Different Scenarios

Scenario	public	protected	default	private
Same Class	✓	✓	✓	✓
Same Package	✓	✓	✓	✗
Subclass (other package)	✓	✓	✗	✗
Other Package (non-subclass)	✓	✗	✗	✗

Interview Tip

- Always prefer least privilege access.
- Use private for fields and provide getters/setters.
- Default access should be used carefully.

Best Practices

- Organize related classes into meaningful packages.
- Use clear package names (e.g., com.company.module).
- Avoid wildcard imports (*), use specific imports.

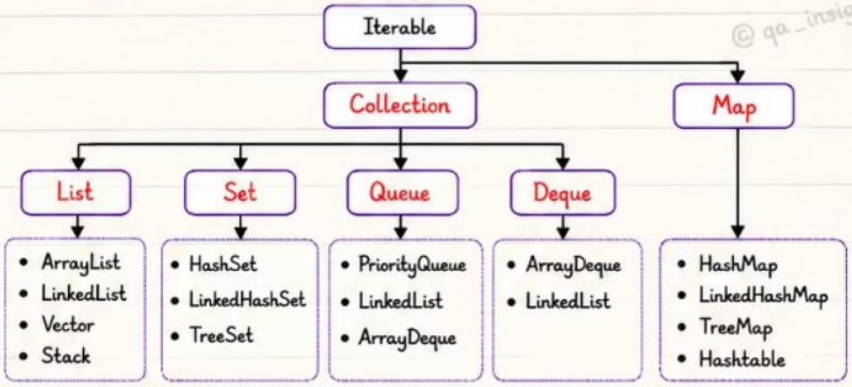
Collections Framework Overview

① What is Collection Framework?

The Collection Framework in Java provides a set of pre-built classes and interfaces to store, manipulate and retrieve groups of objects efficiently.

It is part of java.util package.

② Collection Hierarchy



③ List vs Set vs Queue vs Map

Feature	List	Set	Queue	Map
Definition	Ordered collection of elements.	Collection of unique elements.	Follows FIFO order for processing elements.	Collection of key-value pairs.
Duplicates	Allowed	Not Allowed	Allowed	Keys: No, Values: Yes
Order	Maintains insertion order.	HashSet – No order LinkedHashSet – Insertion order TreeSet – Sorted order	FIFO (First In First Out)	No order (HashMap) Insertion order (LinkedHashMap) Sorted order (TreeMap)
Access	By index (get/set)	No index based access	By head/tail (poll/peek)	By key (get/put)
Example Classes	ArrayList, LinkedList, Vector, Stack	HashSet, LinkedHashSet, TreeSet	PriorityQueue, ArrayDeque, LinkedList	HashMap, LinkedHashMap, TreeMap, Hashtable

④ When to Use Which?

List
- When order matters.
- When duplicate elements are allowed.
- When you need index based access.

Example:
Storing a list of students, names, products.

Set
- When uniqueness is important.
- When you don't need index based access.

Example:
Storing unique IDs, email addresses, visited URLs.

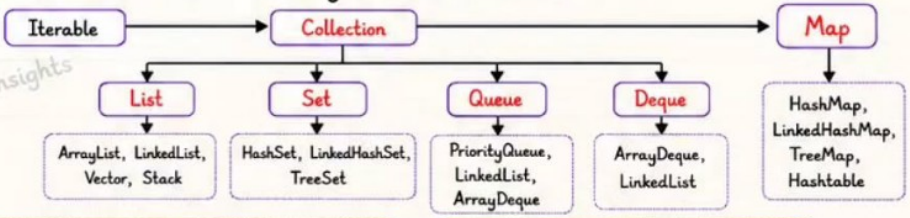
Queue
- When elements should be processed in FIFO order.
- Used in scheduling, buffering, task queues.

Example:
Printer queue, task scheduling.

Map
- When you need to store data as key-value pairs.
- When fast retrieval by key is required.

Example:
Storing userId → user, countryCode → country.

⑤ Collection Framework Diagram (At a Glance)



Remember
- ✓ Collection → single elements.
- ✓ Map → key-value pairs.
- ✓ Choose the right collection for better performance and clean code.

Interview Tip
Understand the differences, use cases and time complexity of each collection class.

Note
All collection classes are non-primitive and work with Objects.

List Implementations in Java

The List interface (in java.util) extends Collection and represents an ordered collection with duplicates.

① List Implementations

ArrayList
- Resizable array implementation of List.
- Implements RandomAccess interface.

Use When?
Fast read access is important and size changes are fewer.

LinkedList
- Doubly linked list implementation.
- Does not implement RandomAccess.

Use When?
Frequent insertions/deletions at beginning or middle.

Vector
- Resizable array, synchronized (thread-safe).
- Legacy class (since Java 1.0).

Use When?
When you need thread-safety but don't want to use Collections.synchronizedList().

Stack
- Extends Vector class.
- Follows LIFO (Last In First Out) order.

Use When?
When you need stack behavior (push/pop/peek).

② Comparison Table (Time Complexity)

Operation	ArrayList	LinkedList	Vector	Stack
Add (end)	O(1) *	O(1)	O(1) * (sync)	O(1)
Add (beginning)	O(n)	O(1)	O(n)	O(n)
Add (middle)	O(n)	O(1)	O(n)	O(n)
Remove (end)	O(1)	O(1)	O(1)	O(1)
Remove (beginning)	O(n)	O(1)	O(n)	O(n)
Remove (middle)	O(n)	O(1)	O(n)	O(n)
Get (by index)	O(1)	O(n)	O(1)	O(n)
Search (contains)	O(n)	O(n)	O(n)	O(n)
Iteration	O(n)	O(n)	O(n)	O(n)

* Amortized O(1). In worst case (resize), O(n).

Interview Tip
- ArrayList is faster for read operations.
- LinkedList is better for add/remove in middle.
- Vector is legacy & synchronized (slower).
- Stack is a legacy class, prefer Deque instead.

Best Practice
- Use ArrayList for most use cases.
- Use LinkedList for frequent insertions/deletions.
- Use Deque (ArrayDeque) instead of Stack.

Remember
List maintains insertion order and allows duplicates. Choose the right implementation based on the operation & performance.

Example - ArrayList

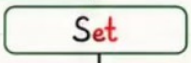
```
List<String> list = new ArrayList<>();
list.add("A");
list.add("B");
list.get(0);        // O(1)
list.add(1, "X");   // O(n)
```

```
add(E e)          -> Add at end
add(int index, E e) -> Add at index
get(int index)    -> Get element
remove(int index) -> Remove element
contains(Object o) -> Search element
```

Set & Queue Framework in Java

Set and Queue are part of java.util package. They store objects efficiently and provide different ordering and access behaviors.

① Set Implementations



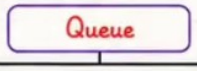
HashSet
- Backed by Hash table.
- No order guaranteed.
- Allows null element.
- Duplicate not allowed.
- O(1) for basic ops.

LinkedHashSet
- Maintains insertion order.
- Backed by Hash table + Doubly Linked List.
- Allows null element.
- Duplicate not allowed.

TreeSet
- Elements sorted in natural order.
- Uses Red-Black Tree.
- No null element.
- Duplicate not allowed.
- O(log n) for basic ops.

Key Points (Set)
- ✓ No duplicates.
- ✓ Unordered (except TreeSet and LinkedHashSet).
- ✓ Used when uniqueness is important.

② Queue Implementations



PriorityQueue
- Elements ordered by priority (natural order / Comparator).
- Not FIFO.
- No null elements.
- O(log n) for insert/remove.

ArrayDeque
- Resizable array implementation.
- Can be used as Queue or Deque.
- Supports add/remove at both ends.
- Null elements not allowed.
- O(1) for basic ops.

Key Points (Queue)
- ✓ Follows FIFO order (except PriorityQueue).
- ✓ Used for task scheduling, buffering, BFS traversal etc.
- ✓ ArrayDeque is faster than LinkedList for Queue.

③ Set vs Queue Comparison

Feature	Set	Queue
Duplicates	Not allowed	Allowed
Order	No order (except LinkedHashSet, TreeSet)	FIFO (except PriorityQueue)
Null Elements	HashSet & LinkedHashSet allow one null, TreeSet doesn't allow null	PriorityQueue & ArrayDeque do not allow null
Underlying Structure	Hash table / Linked List / Red-Black Tree	Heap (PriorityQueue) / Resizable Array (ArrayDeque)
Performance	HashSet ~ O(1), TreeSet ~ O(log n)	PriorityQueue ~ O(log n), ArrayDeque ~ O(1)
Use Cases	Store unique items, membership checking, remove duplicates	Task scheduling, BFS, caching, producer-consumer scenarios

Interview Tip
- Use HashSet for uniqueness.
- Use LinkedHashSet when insertion order matters.
- Use TreeSet when sorted order is required.
- Use PriorityQueue for priority based processing.
- Use ArrayDeque for fast Queue and Deque operations.

Best Practice
- Prefer Set implementations over List when you need uniqueness.
- Prefer ArrayDeque over Stack and LinkedList for Queue operations.

Map Framework in Java

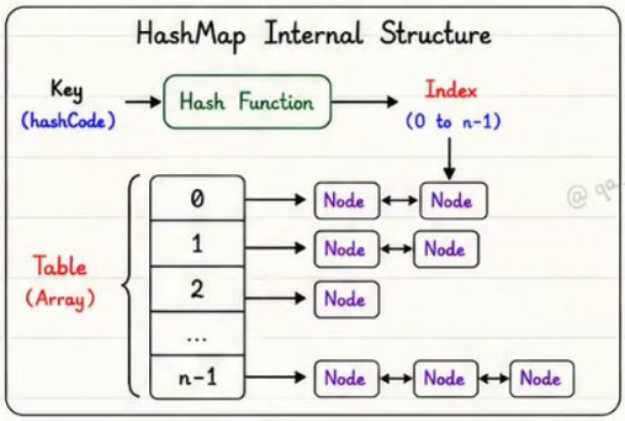
The Map interface in java.util stores data in key-value pairs.
Each key must be unique and maps to exactly one value.

① Map Implementations

- **HashMap**
  - Most common Map implementation.
  - Unordered, allows null key and null values.
  - Not synchronized.
- **LinkedHashMap**
  - Maintains insertion order using doubly linked list.
  - Allows null key and null values.
  - Slightly slower than HashMap.
- **TreeMap**
  - Stores keys in sorted (natural) order.
  - Does not allow null key.
  - Slower than HashMap.
- **Hashtable**
  - Legacy class (Java 1.0).
  - Synchronized and thread-safe.
  - Does not allow null key or null values.

② Internal Working of HashMap

- HashMap uses Hash table (array of buckets).
- Key's hashCode() is used to find index.
- Collisions are handled using Linked List (or Tree in Java 8+).



③ Comparison Table

Feature	HashMap	LinkedHashMap	TreeMap	Hashtable
Order	Unordered	Insertion Order	Sorted by Key (Natural/Comparator)	Unordered
Null Key	Allowed (1 null key)	Allowed (1 null key)	Not Allowed	Not Allowed
Null Value	Allowed	Allowed	Allowed	Not Allowed
Thread Safe	No	No	No	Yes (Synchronized)
Performance	Fastest	Slightly slower than HashMap	Slow (Tree operations O(log n))	Slow (Synchronized)
Underlying Data Structure	Hash Table + Linked List / Tree	Hash Table + Doubly Linked List	Red-Black Tree	Hash Table + Linked List
Class	java.util.HashMap	java.util.LinkedHashMap	java.util.TreeMap	java.util.Hashtable

Interview Tip

- Use HashMap when you don't care about order and need best performance.
- Use LinkedHashMap when insertion order is important.
- Use TreeMap when you need sorted keys.
- Avoid Hashtable in modern applications.

Best Practice

- Prefer HashMap over Hashtable.
- If keys need ordering, use TreeMap.
- If insertion order is required, use LinkedHashMap.
- Avoid storing mutable objects as keys.

Example - HashMap

```
Map<String, Integer> map = new HashMap<>();
map.put("A", 10);
map.put("B", 20);
map.put(null, 30);        // null key allowed
map.put("A", 40);         // value updated
System.out.println(map); // {A=40, B=20, null=30}
```

Common Methods

put(key, value)	→ Add/Update
get(key)	→ Get value
remove(key)	→ Remove key
containsKey(key)	→ Check key
size()	→ Number of entries

Generics & Wrapper Classes

① What are Generics?

Generics allow us to write classes, interfaces and methods with Type Parameters.
It provides type safety and reusability.

```
class Box<T> {                          // Type Parameter T
  private T value;
  Box(T value) { this.value = value; }
  T getValue() { return value; }
}

Box<String> b = new Box<>("Hello");
String s = b.getValue();                // Type safe
```

② Type Safety

Generics enforce compile-time type checking,
It prevents ClassCastException at runtime.

Without Generics	With Generics

`List list = new ArrayList();`
`list.add("Java");`
`list.add(10);`
`String s = (String) list.get(0);`
`// Risk of ClassCastException`
	`List<String> list =`
	`  new ArrayList<>();`
	`list.add("Java");`
	`// list.add(10); ✗`
	`// Compile time error`

③ Wrapper Classes

Wrapper classes provide objects for primitive data types.

Primitive	Wrapper Class
byte	Byte
short	Short
int	Integer
long	Long
float	Float
double	Double
char	Character
boolean	Boolean

④ Auto Boxing

Converting primitive → wrapper class automatically by the compiler.

```
int a = 10;
Integer i = a;  // auto boxing
```

int (primitive) —auto boxing→ Integer (object)

⑤ Auto Unboxing

Converting wrapper class → primitive automatically by the compiler.

```
Integer i = 20;
int a = i;     // auto unboxing
```

Integer (object) —auto unboxing→ int (primitive)

⑥ Advantages of Generics & Wrapper Classes

- ✓ Type Safety – Catches errors at compile time.
- ✓ Code Reusability – Write once, use with any data type.
- ✓ No Type Casting – Reduces ClassCastException.
- ✓ Better Performance – Generics work with compile time types.
- ✓ Collection Framework – Works only with objects (wrapper classes).
- ✓ Utility Methods – Wrapper classes provide useful methods like parseInt(), valueOf(), etc.

Interview Tip

- Generics provide compile-time type safety.
- Wrapper classes are immutable.
- Use valueOf() instead of new for better performance (caching).
- Know auto boxing/unboxing well.

⚠ Common Mistake

Using raw types (without generics) defeats the purpose of generics.

Avoid: `List list = new ArrayList();  // Raw type – unsafe`

Prefer: `List<String> list = new ArrayList<>();  // Type safe`

Remember

Collections can store only objects, not primitives. Use wrapper classes. (int → Integer, boolean → Boolean)

Comparable vs Comparator in Java

Used to define the natural ordering (Comparable) or custom ordering (Comparator) for objects so that they can be sorted.

① Comparable Interface

- Part of java.lang package.
- Defines natural ordering of objects.
- Class must implement Comparable<T>.
- Only one natural ordering possible per class.
- Uses compareTo() method.

```
public interface Comparable<T> {
    int compareTo(T obj);
}
```

② Comparator Interface

- Part of java.util package.
- Defines custom ordering of objects.
- Class does NOT need to implement anything.
- Multiple comparators can be created.
- Uses compare() method.

```
public interface Comparator<T> {
    int compare(T o1, T o2);
}
```

③ compareTo() vs compare()

compareTo()	compare()
• Method of Comparable interface.	• Method of Comparator interface.
• Used for natural ordering.	• Used for custom ordering.
• Single parameter (object).	• Two parameters (o1, o2).
• Used by sort() in Collections.	• Passed explicitly in sort() or sorted().

④ Custom Object Sorting

Example : Sort list of Student by age (natural order) and by name (custom order).

Natural Order (Comparable)

```
class Student implements
Comparable<Student> {
  int age; String name;
  public int compareTo(Student s) {
    return Integer.compare(this.age,
                           s.age);
  }
}
Collections.sort(list);
```

Custom Order (Comparator)

```
Comparator<Student> nameComp =
  new Comparator<Student>() {
    public int compare(Student s1,
                       Student s2) {
      return s1.name.compareTo(
                       s2.name);
    }
};
Collections.sort(list, nameComp);
```

⑤ Comparison Table

Feature	Comparable	Comparator
Package	java.lang	java.util
Method	compareTo(T obj)	compare(T o1, T o2)
Parameters	One (object)	Two (objects)
Sorting	Natural Ordering	Custom Ordering
Implementation	Class must implement Comparable	Separate Comparator class / Lambda
Multiple Orders	Only one	Multiple
Used By	Collections.sort(list)	Collections.sort(list, comparator) list.sort(comparator)
Performance	Slightly better	Slightly slower (extra object)

Interview Tip

- Use Comparable when natural order exists and won't change.
- Use Comparator when multiple sorting logic is required.
- TreeSet and TreeMap use Comparable or Comparator.

Best Practice

- Prefer compareTo() over subtraction to avoid overflow.
- Return Integer.compare(a, b) instead of a - b.
- Keep comparator logic consistent and transitive.

Remember

If a class implements both Comparable and equals(), ensure they are consistent.
Consistent with equals: a.equals(b) == true → a.compareTo(b) == 0

Java 8 Features

Java 8 introduced several powerful features that make code more concise, readable and functional.

① Lambda Expressions

- A lambda expression is a short block of code that takes parameters and returns a value.
- Helps in writing concise implementations.

```
// Traditional way
Runnable r = new Runnable() {
    public void run() { System.out.println("Hi"); }
};
// Lambda way
Runnable r = () -> System.out.println("Hi");
```

② Functional Interface

- An interface with exactly one abstract method.
- Can be used as the target type for lambda.

```
@FunctionalInterface
interface MyFunctionalInterface {
    void execute(String msg);   // only one abstract method
}

MyFunctionalInterface obj = msg -> System.out.println(msg);
```

③ Method References

- Shorthand notation to refer to a method without executing it.

Type	Example	Equivalent Lambda
Static Method	ClassName::staticMethod	(args) -> ClassName.staticMethod(args)
Instance Method	obj::instanceMethod	(args) -> obj.instanceMethod(args)
Constructor	ClassName::new	(args) -> new ClassName(args)
Instance Method of Type	ClassName::instanceMethod	(obj, args) -> obj.instanceMethod(args)

④ Default Methods

- Interfaces can have methods with implementation.
- Helps in backward compatibility.

```
interface MyInterface {
    default void show() {
        System.out.println("Default Method");
    }
}
MyInterface obj = () -> {};   // Lambda
obj.show();      // Works fine
```

⑤ Static Methods in Interface

- Interfaces can have static methods.
- Called using Interface name.

```
interface MathUtils {
    static int add(int a, int b) {
        return a + b;
    }
}
int sum = MathUtils.add(10, 20);  // 30
```

⑥ Optional Class

- Used to avoid NullPointerException.
- Represents optional value.

```
Optional<String> name = Optional.of("QA Insights");
String len = name.map(String::length)
                 .orElse(0);    // 12

Optional<String> empty = Optional.empty();
String value = empty.orElse("Not Present");
```

Interview Tip

- Lambda = Anonymous function.
- Functional Interface + Lambda = Functional Programming.
- Method Reference improves readability.
- Optional is a container that may or may not contain a value.

Best Practice

- Prefer lambda for short code.
- Use method references for better readability.
- Avoid nulls, use Optional.
- Use default and static methods in interface to enhance extensibility.

Remember

Lambda + Stream API = Powerful combination for data processing.

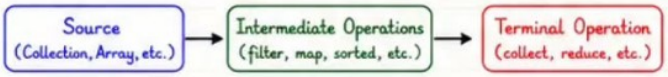
Example: `list.stream().filter(e -> e.active()).map(User::getName).toList();`

Stream API in Java

Stream API (java.util.stream) allows functional-style operations on collections.

① Stream Pipeline

A Stream pipeline consists of a source, zero or more intermediate operations and a terminal operation.



Key Points

- Intermediate operations return a Stream.
- Terminal operations produce a result or side-effect.
- Streams are lazy: intermediate ops are executed only when a terminal op is invoked.

② Common Intermediate Operations

- filter(Predicate)
  → Filters elements based on a condition.
- map(Function)
  → Transforms each element to another element.
- sorted()
  → Returns elements in natural sorted order.
- distinct()
  → Removes duplicate elements.

③ Common Terminal Operations

- collect(Collector) → Collects the result into a collection.
- reduce(identity, accumulator) → Combines elements into a single result.
- forEach(Consumer) → Performs an action for each element.
- count() → Returns the count of elements.
- anyMatch(), allMatch(), noneMatch()
  → Match operations returning boolean.

④ Example: Stream Pipeline in Action

```
List<String> names = List.of("John","Alice","bob","David","alice");
List<String> result = names.stream()
        .filter(n -> n.length() > 3)      // Intermediate
        .map(String::toUpperCase)         // Intermediate
        .distinct()                       // Intermediate
        .sorted()                         // Intermediate
        .collect(Collectors.toList());    // Terminal
System.out.println(result);
// Output: [ALICE, BOB, DAVID, JOHN]
```

⑤ Intermediate vs Terminal Operations

Feature	Intermediate Operations	Terminal Operations
Returns	Stream	Result / Collection / Primitive / void
Execution	Lazy (executed only when terminal op is invoked)	Eager (triggers the execution of pipeline)
Examples	filter(), map(), sorted(), distinct()	collect(), reduce(), forEach(), count()
Purpose	Transform or filter data	Produce result or side-effect
Can be chained	Yes (multiple times)	No (only one at the end)

⑥ reduce() Example

```
List<Integer> numbers = List.of(1, 2, 3, 4, 5);
int sum = numbers.stream()
        .reduce(0, (a, b) -> a + b);   // identity = 0
System.out.println("Sum: " + sum);     // Sum: 15
```

⑦ collect() Example

```
List<String> names = List.of("a","b","c");
Set<String> set = names.stream()
        .collect(Collectors.toSet());
System.out.println(set); // [a, b, c]
```

Interview Tip

- Stream is not a data structure, it is a function pipeline.
- Intermediate ops are lazy.
- Use Streams for clean, readable code.

Best Practice

- Prefer Stream API for bulk operations.
- Avoid side-effects inside Stream.
- Use method references for better readability.

Common Mistake

- Forgetting terminal operation (nothing executes).
- Modifying source collection during stream.
- Using parallelStream() without need.

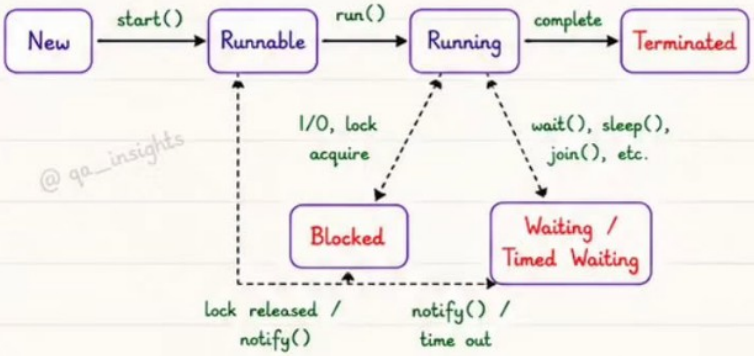
Multithreading in Java

Multithreading allows multiple threads to run concurrently within the same process, improving performance and responsiveness.

① Process vs Thread

Process	Thread
• Independent program in execution.	• Smallest unit of execution.
• Has its own memory space.	• Shares memory space with other threads.
• Creation is heavy and resource intensive.	• Creation is lightweight and fast.
• Communication is done using IPC.	• Communication is easy via shared memory.

② Thread Lifecycle



③ Thread Class

Create a thread by extending "Thread" and overriding "run()".

```
class MyThread extends Thread {
    public void run() {
        System.out.println("Thread is running");
    }
}
MyThread t = new MyThread();
t.start();     // starts the thread
```

④ Runnable Interface

Create a thread by implementing "Runnable" interface.

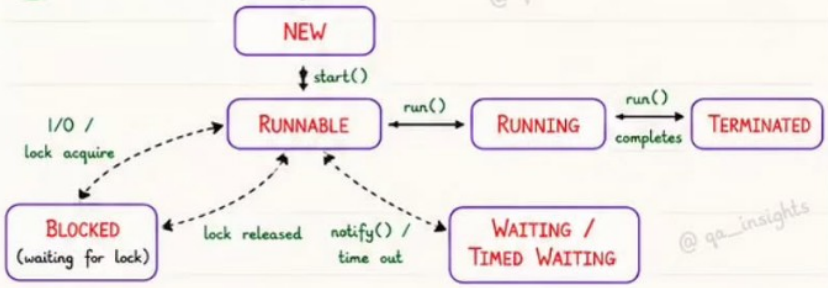
```
class MyRunnable implements Runnable {
    public void run() {
        System.out.println("Thread is running");
    }
}
Thread t = new Thread(new MyRunnable());
t.start();     // starts the thread
```

⑤ Synchronization

Used to control access to shared resources and prevent "race condition".

```
class Counter {
    private int count = 0;
    public synchronized void increment() {
        count++;
    }
    public synchronized int getCount() {
        return count;
    }
}
```

⑥ Thread States Diagram



- NEW : Thread is created but not started.
- RUNNABLE : Waiting for CPU time.
- RUNNING : Actively executing.
- BLOCKED : Waiting to acquire a lock.
- WAITING : Waiting indefinitely for another thread to perform an action.
- TIMED WAITING : Waiting for a specified time.
- TERMINATED : Execution completed.

Interview Tip

- start() creates a new thread.
- run() is the entry point.
- sleep() doesn't release lock.
- yield() suggests thread switch.

Best Practices

- Avoid calling sleep() and yield() for synchronization.
- Prefer ExecutorService for thread management in real applications.
- Keep synchronized blocks small.

Common Mistakes

- Calling run() instead of start().
- Not handling InterruptedException.
- Synchronizing on String objects.
- Creating too many threads.

Executor Framework in Java

Executor Framework provides a way to manage and reuse threads efficiently using thread pools and asynchronous tasks.

① Executor

The Executor interface is the root of the executor framework. It has a single method.

```
public interface Executor {
    void execute(Runnable command);
}
```

↳ Used to execute Runnable tasks.

② ExecutorService

ExecutorService is a subinterface of Executor. It adds features like managing lifecycle and returning results.

```
public interface ExecutorService
    extends Executor {
  void shutdown();
  List<Future<?>> invokeAll(...);
  <T> Future<T> submit(Callable<T> task);
}
```

↳ Provides methods to control threads and get results.

③ Callable

Callable is similar to Runnable but it can return a result and throw checked exceptions.

```
@FunctionalInterface
public interface Callable<V> {
  V call() throws Exception;
}
```

↳ Used when a task returns a result.

④ Future

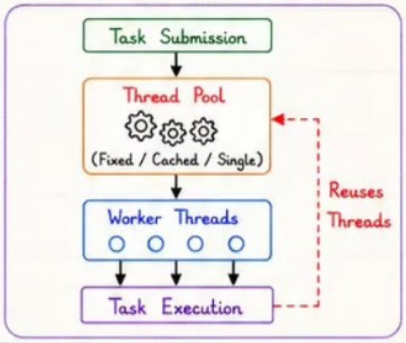
Future represents the result of an asynchronous computation.

```
Future<String> future =
    executorService.submit(() -> "Hello");
String result = future.get();  // blocks
boolean done = future.isDone();
```

↳ Methods: get(), isDone(), cancel(), isCancelled().

⑤ Thread Pool

A Thread Pool reuses a fixed number of threads to execute multiple tasks.



⑥ Types of Thread Pools

Factory Method	Description
newFixedThreadPool(n)	Fixed number of threads.
newCachedThreadPool()	Creates new threads as needed, reuses idle threads.
newSingleThreadExecutor()	Single worker thread.
newScheduledThreadPool(n)	For scheduled / delayed tasks.

⑦ Example: Executor Framework in Action

```
ExecutorService executor = Executors.newFixedThreadPool(2);   // Create thread pool with 2 threads
Callable<Integer> task = () -> {                               // Define a Callable task
    Thread.sleep(1000);                                        // Simulate task
    return 42;                                                 // Return result
};
Future<Integer> future = executor.submit(task);                // Submit task
System.out.println("Result: " + future.get());  // 42         // Get result (blocks until done)
executor.shutdown();                                           // Shutdown the pool
```

Best Practices

- ✓ Always shutdown ExecutorService using shutdown() or shutdownNow().
- ✓ Use appropriate thread pool size based on workload.
- ✓ Avoid creating too many threads.
- ✓ Handle InterruptedException properly.
- ✓ Prefer Callable + Future when a result is needed.

Key Points

- Executor is the root interface for task execution.
- ExecutorService provides advanced lifecycle management.
- Callable returns result; Runnable does not.
- Future represents the result of asynchronous computation.
- Thread pools improve performance by reusing threads.

Interview Tip

- When you need a result → use Callable.
- When you just want to run a task → use Runnable.
- Use ExecutorService over raw threads.
- Thread pool = Better performance + resource control.

Common Mistakes

- Forgetting to shutdown ExecutorService.
- Ignoring Exception from Future.get().
- Creating new threads instead of using thread pools.

File Handling & Serialization in Java

Java provides classes to work with files and to store objects permanently using Serialization.

① File Handling Classes

File Class

- Represents a file or directory path.
- Provides methods to create, delete, check existence, etc.

```
File f = new File("test.txt");
f.createNewFile();
f.exists();
f.delete();
```

FileReader

- Reads character data from a file.
- Used with simple character input.

```
FileReader fr =
 new FileReader("a.txt");
int ch;
while((ch = fr.read())!=-1)
 System.out.print((char)ch);
fr.close();
```

FileWriter

- Writes character data to a file.
- Creates file if it does not exist.

```
FileWriter fw =
new FileWriter("a.txt");
fw.write("Hello Java");
fw.flush();
fw.close();
```

BufferedReader

- Reads text efficiently line by line.
- Uses internal buffer for better performance.

```
BufferedReader br =
 new BufferedReader(
  new FileReader("a.txt"));
String line;
while((line = br.readLine())
 != null) System.out.println(line);
br.close();
```

BufferedWriter

- Writes text efficiently line by line.
- Uses internal buffer for better performance.

```
BufferedWriter bw =
 new BufferedReader(
  new FileWriter("a.txt"));
bw.write("Hello Java");
bw.newLine();
bw.close();
```

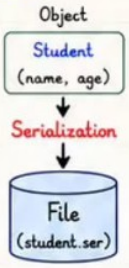
② Serialization

Serialization is the process of converting an object's state into a byte stream so that it can be saved to a file or sent over network.

Steps:

1. Object must implement Serializable interface.
2. Use ObjectOutputStream to write object to file.

```
class Student implements Serializable {
  String name;  int age;
  Student(String n, int a){ name=n; age=a; }
}
Student s = new Student("Mahesh", 25);
FileOutputStream fos = new FileOutputStream("student.ser");
ObjectOutputStream oos = new ObjectOutputStream(fos);
oos.writeObject(s);   // serialization
oos.close();  fos.close();
```



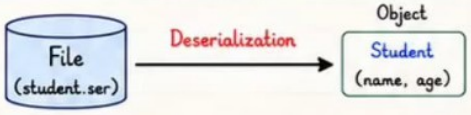
③ Deserialization

Deserialization is the reverse process of Serialization. It converts the byte stream back into an object.

Steps:

1. Use ObjectInputStream to read object from file.
2. Cast the object to original class type.

```
FileInputStream fis = new FileInputStream("student.ser");
ObjectInputStream ois = new ObjectInputStream(fis);
Student s2 = (Student) ois.readObject();   // deserialization
System.out.println(s2.name + " " + s2.age);
ois.close();  fis.close();
```



④ Key Points

- File class is used to perform file & directory operations.
- Use buffered classes (BufferedReader/Writer) for better performance.
- Serialization is used to persist object state.
- Deserialization recreates the object from stored state.
- Serialized classes must implement Serializable.
- transient and static fields are not serialized.

Interview Tip

- Difference between File vs Path.
- When to use BufferedReader over FileReader?
- Why serialization is used?
- What happens if serialVersionUID changes?
- Difference between Serializable and Externalizable.

Best Practices

- Always close streams in finally block or use try-with-resources.
- Avoid serialization when not required.
- Use buffered streams for large files.
- Validate file paths and handle exceptions.

Remember

Serialization = Object → Byte Stream (Save)

Deserialization = Byte Stream → Object (Load)

Common Exceptions

- FileNotFoundException – File not found.
- IOException – General I/O failure.
- ClassNotFoundException – Class not found during deserialization.

JVM Memory Management

JVM manages memory areas to store data, objects, class information and execution context. Proper memory management ensures performance and stability.

① Heap Memory

- Stores Objects and Arrays.
- Shared by all threads.
- Created when JVM starts.
- Subject to Garbage Collection.

```
Person p = new Person(); // Object in Heap
String s = new String("Hello");
```

② Stack Memory

- Stores method calls, local variables and references.
- Each thread has its own Stack.
- LIFO (Last In First Out).
- Automatically managed.

```
public void show(int x) {  // Stack frame
    int y = 10;            // local variable
    show(y);               // method call
}
```

③ Method Area (Class Area)

- Stores class metadata, constants, static variables and method info.
- Shared by all threads.
- Exists as long as JVM is running.

```
class MyClass {
    static int count = 0;   // in Method Area
    void method() {}
}
```

④ Metaspace (Permanent Generation Replacement)

- Stores class metadata (from Java 8+).
- Replaces Permanent Generation.
- Allocated from native memory (not in heap).

```
Class<?> clazz = MyClass.class;  // metadata in Metaspace
```

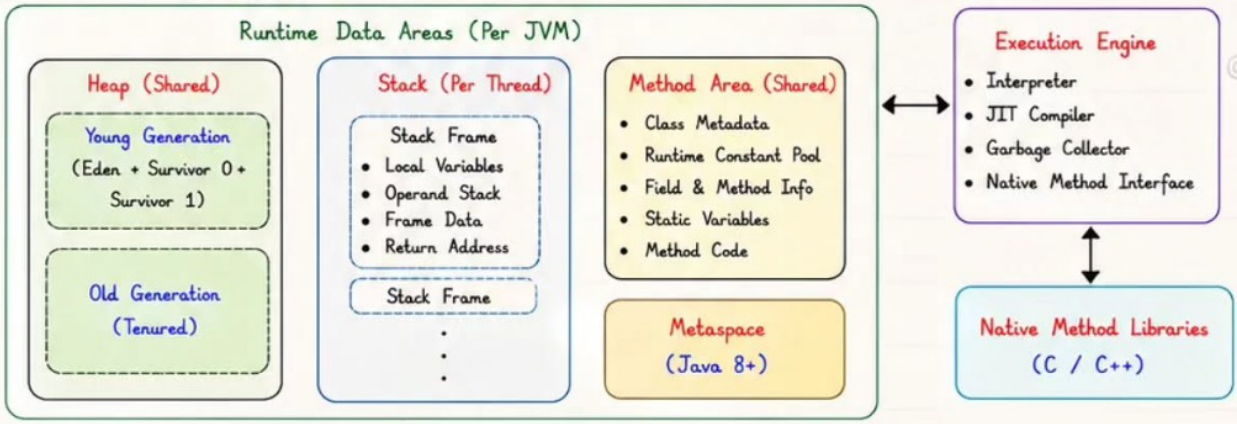
⑤ Garbage Collection (GC)

- Automatically reclaims memory of unreferenced objects.
- Improves performance and prevents memory leaks.
- Types: Serial GC, Parallel GC, G1 GC, ZGC.

GC Goals

- ✓ Reclaim unused memory
- ✓ Reduce memory usage
- ✓ Improve application performance

⑥ JVM Memory Diagram



Key Points

- Heap → Stores objects, managed by GC.
- Stack → Stores method calls, automatic management.
- Method Area & Metaspace → Store class info.
- GC → Removes unreferenced objects.
- More heap → For objects, more stack → For threads.

Interview Tip

- What is difference between Stack and Heap?
- Why Metaspace introduced in Java 8?
- Which GC is default in Java 17? (G1 GC)
- How does Garbage Collection work?
- What causes OutOfMemoryError?

Best Practices

- Make objects eligible for GC quickly.
- Avoid memory leaks (close resources).
- Tune heap size: -Xms, -Xmx.
- Use profilers to monitor memory.

Advanced Java Concepts

These concepts are frequently asked in interviews and used in real-world applications.

① Reflection API

Reflection allows inspecting and modifying classes, methods, fields at runtime.

```
Class<?> clazz = Class.forName("Student");
Object obj = clazz.getDeclaredConstructor().newInstance();
Method m = clazz.getMethod("setName", String.class);
m.invoke(obj, "Mahesh");
Field f = clazz.getDeclaredField("age");
f.setAccessible(true);
f.setInt(obj, 25);
```

Use Cases

- Frameworks
- Dependency Injection
- Dynamic class loading

② Annotations

Annotations provide metadata to the compiler or runtime. They don't change program logic.

```
@Override
public String toString() {
    return "Student";
}
@Deprecated
public void oldMethod() {
    // do something
}
```

Built-in Annotations

- @Override
- @Deprecated
- @SuppressWarnings
- @FunctionalInterface
- @Retention, @Target

③ Immutable Class

Immutable class cannot be changed once an object is created. All fields are final and no setters.

```
final class Person {
    private final String name;
    private final int age;
    public Person(String name, int age) {
        this.name = name; this.age = age;
    }
    public String getName() { return name; }
    public int getAge() { return age; }
}
```

Advantages

- Thread-safe
- Secure
- Can be used as Map keys
- Easy to cache

④ Singleton Pattern

Ensures only one instance of a class exists and provides a global access point.

```
class Singleton {
    private static Singleton instance;
    private Singleton() { }
    public static Singleton getInstance() {
        if(instance == null)
            instance = new Singleton();
        return instance;
}
```

Types

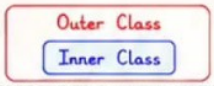
- Eager Initialization
- Lazy Initialization
- Double Checked Locking
- Enum Singleton

Best way: Use Enum Singleton.

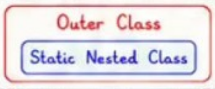
⑤ Inner Classes

A class defined inside another class.

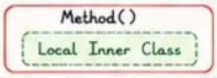
- Non-static Inner Class
  - Has access to outer class members.



- Static Nested Class
  - Behaves like a normal class.
  - Can't access non-static members of outer class.



- Local Inner Class
  - Defined inside a method.
  - Accessible only within that method.



- Anonymous Inner Class
  - No name, used for one-time use.
  - Common with Interfaces/Abstract classes.

```
new Interface() {
    // implementation
};
```

⑥ Enums

Enum is a special data type that represents a group of constants (immutable).

```
enum Day {
    MONDAY, TUESDAY, WEDNESDAY,
    THURSDAY, FRIDAY, SATURDAY, SUNDAY;
}

Day d = Day.MONDAY;
System.out.println(d);      // MONDAY
```

Features

- Type-safe
- Implicitly extends java.lang.Enum
- Can have fields, constructors & methods
- Used in switch statements

Key Points

- Use Reflection for dynamic operations.
- Use Annotations for configuration & metadata.
- Immutable objects are safer and thread-safe.
- Singleton controls object creation.
- Inner classes improve code organization.
- Enums are better than int constants.

Interview Tips

- Difference between static nested and inner class?
- How does Singleton break using reflection? (Use enum to prevent)
- Why String class is immutable?
- When to use Enum vs Class constants?

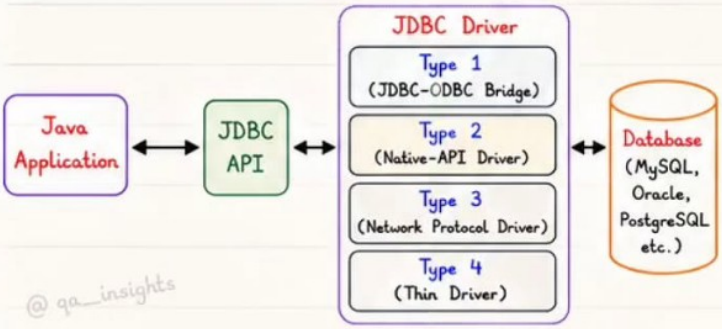
Best Practices

- Prefer composition over deep nesting.
- Avoid reflection unless necessary.
- Make classes immutable whenever possible.
- Use Enum Singleton for thread safety.
- Document custom annotations clearly.

JDBC Fundamentals

JDBC (Java Database Connectivity) is an API that allows Java applications to connect and interact with databases.

① JDBC Architecture



② DriverManager

- DriverManager class manages the list of database drivers.
- It is used to establish a connection.
- It loads the appropriate driver and creates the connection.

```
Connection conn = DriverManager.getConnection(
    "jdbc:mysql://localhost:3306/qa_db",
    "root", "password");
```

③ Connection

- Connection interface represents a session with the database.
- It is created using DriverManager.
- Used to create Statement, PreparedStatement or CallableStatement.

```
Connection conn = DriverManager.getConnection(
    "jdbc:mysql://localhost:3306/qa_db", "root", "password");
// Don't forget to close the connection
```

④ Statement

- Statement is used to execute simple SQL queries.
- Not safe from SQL Injection.
- Suitable for static SQL queries.

```
Statement stmt = conn.createStatement();
String sql = "SELECT * FROM users";
ResultSet rs = stmt.executeQuery(sql);
```

⑤ PreparedStatement

- Precompiled SQL statements.
- Parameterized queries.
- Protects from SQL Injection and improves performance.

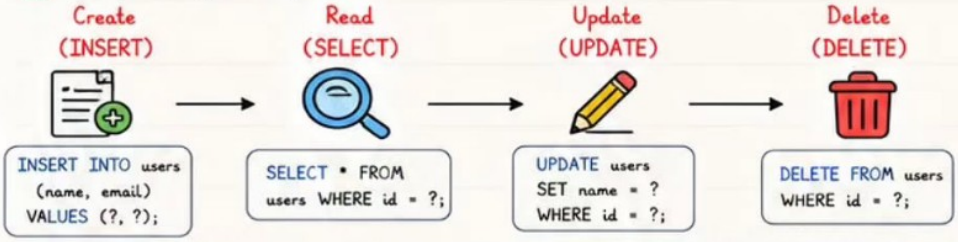
```
PreparedStatement ps = conn.prepareStatement(
   "SELECT * FROM users WHERE id = ?");
ps.setInt(1, 101);
ResultSet rs = ps.executeQuery();
```

⑥ ResultSet

- ResultSet represents the result of a query.
- Provides a cursor to iterate through rows.

```
while (rs.next()) {
    int id = rs.getInt("id");
    String name = rs.getString("name");
    System.out.println(id + " - " + name);
}
```

⑦ CRUD Workflow Diagram



Key Points

- Load the driver (auto in JDBC 4+).
- Get a Connection.
- Create Statement / PreparedStatement.
- Execute SQL query.
- Process ResultSet.
- Close resources.

Interview Tip

- What is JDBC? → API to connect Java with DB.
- Difference between Statement & PreparedStatement?
- Why close Connection, Statement, ResultSet?

Best Practices

- Use PreparedStatement for security & performance.
- Always close resources (try-with-resources). (Connection, Statement, ResultSet)
- Use meaningful exception handling.

Common Mistakes

- Forgetting to close resources.
- Using Statement for user input (SQL Injection).
- Not handling SQL Exceptions.